carla-expert-nmpc: architecture

SimLingo through an urban NMPC on Bench2Drive: how the parts fit, what each one computes, and every parameter. Built from commit d320b04 + uncommitted changes; the parameter tables are read from the code.

1. The stack

One agent drives Bench2Drive's ego. **SimLingo** (a vision-language driving model) looks at the cameras and predicts a path and timed speed waypoints; its own PID used to follow them. Here the **urban NMPC** follows them instead, inside the constraints a **world model** gives: the actors it must keep clear of, the lane, the lights and signs. The world model reads CARLA's ground truth for now, but only what a perception stack on Bench2Drive's sensors could see, and reports it relative to the car, so a learned perception model can take its place behind the same Scene.

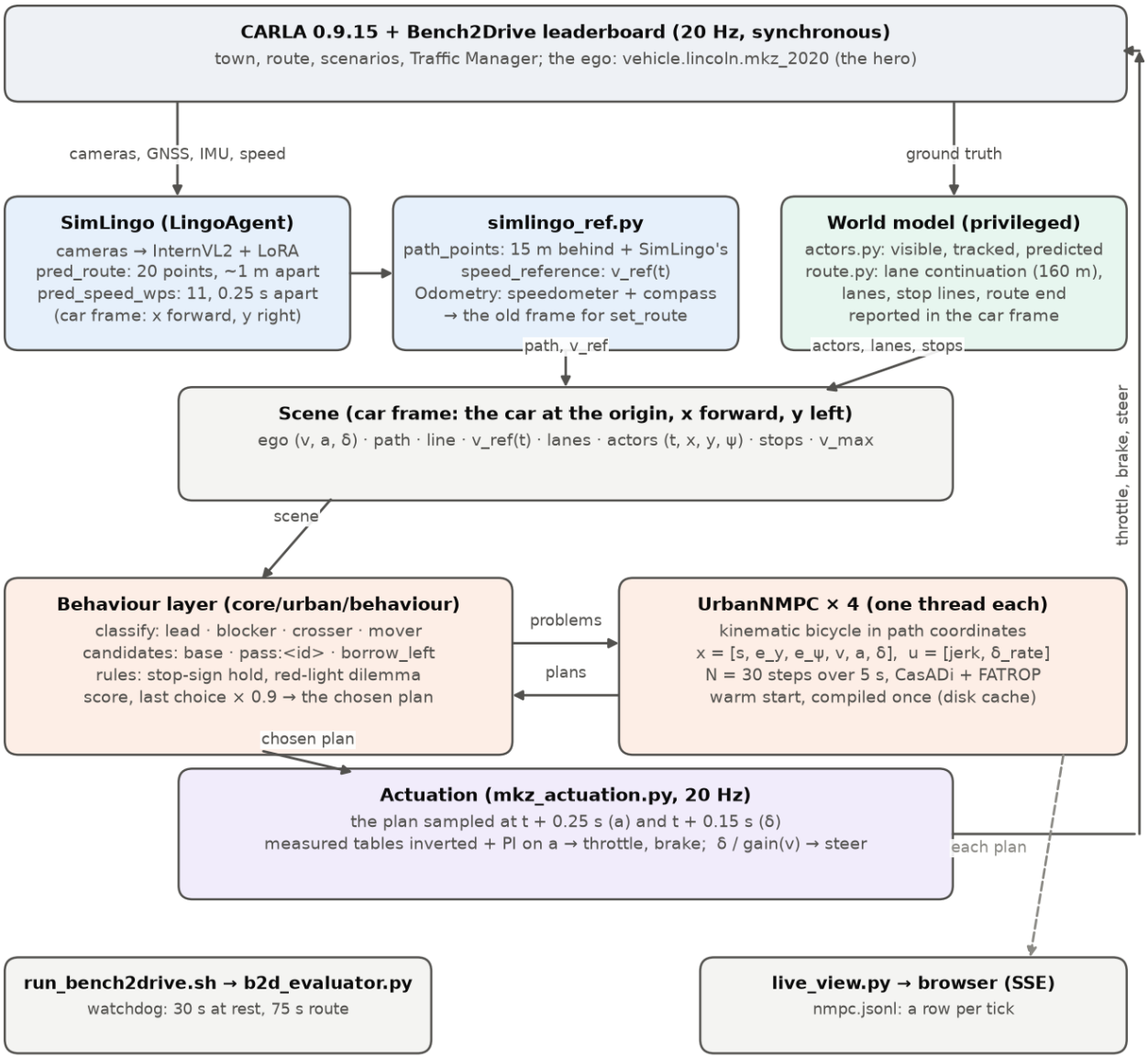


Figure 1. Data flow through one plan. Blue: SimLingo and its reference; green: the world model (privileged); orange: the behaviour layer and the NMPC (C++, core/urban); violet: actuation; grey: the launcher, logging and the live view, which do not take part in driving.

Part	Where	What it does
SimLingo	`external/simlingo/team_code/agent_simlingo.py`	LingoAgent: predicts pred_route and pred_speed_wps each tick

Part	Where	What it does
Agent	`expert/agents/simlingo_nmpc_agent.py`	LingoAgent with control_pid replaced: the per-tick loop, scene, actuation, log
Reference	`expert/simlingo_ref.py`	SimLingo's outputs as path points and v_ref(t); odometry between plans
World model	`expert/world_model/actors.py, route.py`	visible actors with predictions; lanes, stop lines, route end, lane continuation
Frames	`expert/frames.py`	CARLA (left-handed, degrees) to the library's frame; Frame: world to car frame
Behaviour	`core/urban/{include,src}/behaviour`	classification, traffic rules, candidates, parallel solves, choice
NMPC	`core/urban/{include,src}/urban_nmpc`	the optimal control problem, CasADi + FATROP, warm start, set_route
Path	`core/urban/.../path`	biarc path through waypoints: projection, pose, curvature
Bindings	`bindings/expertctl.cpp`	the Python module expertctl (pybind11, Python 3.8)
Actuation	`expert/mkz_actuation.py`	acceleration and road-wheel angle to throttle, brake, steer (measured tables)
Launcher	`scripts/run_bench2drive.sh, b2d_evaluator.py`	one leaderboard process per route; watchdog; results
Live view	`expert/live_view.py, .html`	each plan pushed to a browser page (SSE)
Offline tests	`sim/urban/scenarios.cpp, tests/`	22 closed-loop scenarios without CARLA (11 × 2), pytest

2. One tick

The leaderboard ticks CARLA at 20 Hz in synchronous mode and waits for the agent each tick. SimLingo predicts every tick. Every second tick the agent builds a new path from the latest prediction, reads the world model, and runs a behaviour step (10 Hz). Every tick it samples the current plan a little ahead, for the car's lags, and turns it into throttle, brake and steer.

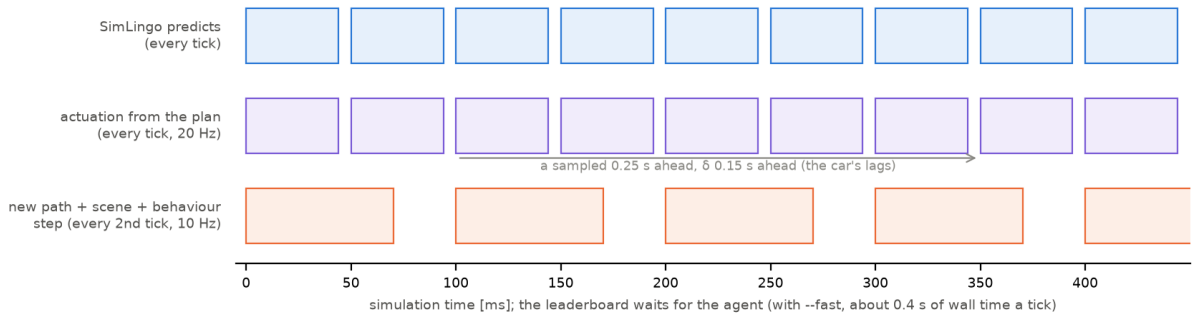


Figure 2. Timing within a tick and across ticks.

Per plan, in order (SimLingoNMPCAgent.control_pid):

- Odometry: the speedometer and the compass's change since the last tick move the car in the last plan's frame.
- Path: SimLingo's 20 points, y flipped to the left, 15 m of straight line prepended behind the car, then 160 m along the map's lane past the last point (route.py). Built as a biarc Path.
- set_route(path, old frame): the last plan carried into the new path's coordinates through the old frame's pose in the new one, so the solver starts warm.
- World model: actors detected and tracked (in the world), predicted 5 s ahead, reported in the car's frame; lanes and stop lines along the path; v_max from the speed limit.
- Scene → Behaviour.step → Decision (the chosen plan, all candidates, notes), then the log row and the live view.

3. Frames: why nothing needs a global position

SimLingo predicts in the car's frame at the moment of the prediction, so the NMPC plans in that frame: the car at the origin, x forward, y left. The car's state is what it measures: speed (speedometer), acceleration (IMU, low-passed), road-wheel angle (from the last steer command). Between two plans

the frame moves with the car; odometry (speed × dt, compass change) gives the old frame's pose in the new one for the warm start. Only the world model touches the world: its ground truth is placed in the car's frame through the car's true pose (frames.Frame), which is what a sensor on the car would report.

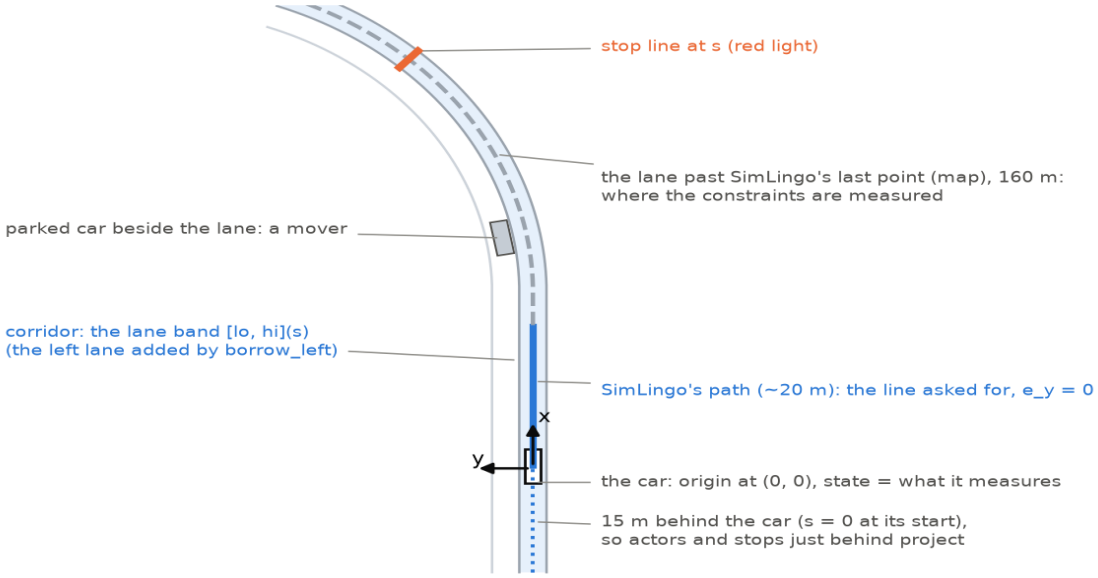


Figure 3. The path and what lies along it, in the car's frame. Path coordinates: s along the path from its start 15 m behind the car (so s ≈ 15 m at the car), e_y to the left of it.

An earlier version placed SimLingo's path in the world through SimLingo's own GNSS/UKF pose. On Town12 that frame was 497.6 m off CARLA's, because SimLingo estimates its GNSS reference itself, so every actor and lane came out 497 m away. Planning in the car's frame removes that class of error.

4. The NMPC

A kinematic bicycle in the path's coordinates, solved as one nonlinear program per candidate:

```
state x = [s, e_y, e_ψ, v, a, δ]; input u = [jerk, δ_rate]
β = atan(b tan δ / L) (the sideslip at the centre of gravity)
s′ = v cos(e_ψ + β) / (1 − κ(s) e_y)
e_y′ = v sin(e_ψ + β)
e_ψ′ = v cos β tan δ / L − κ(s) s′
v′ = a; a′ = jerk; δ′ = δ_rate
```

κ(s) enters as parameters: the path's mean curvature over each step of the shifted previous plan, so the compiled solver does not depend on the route and one compiled solver serves every path. RK4 integration per step; the steps are 0.1 s for the first second and 0.2 s after.

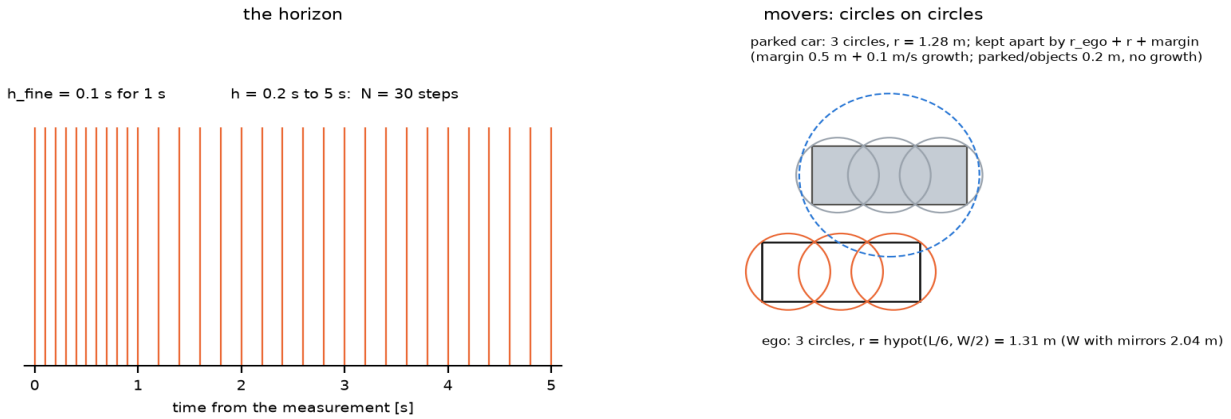


Figure 4. Left: the horizon's steps. Right: how bodies are kept apart.

Cost per step, each weighted by its step's length (w_* in Table 4):

Term	Expression	Why
speed	$(v - v_ref)^2$	v_ref: SimLingo's speed, capped by the limit, tapered to stops, leads and bends
lane	$(e_y - e_ref)^2$	e_ref: SimLingo's path (0), kept inside the lane band; the left lane centre when borrowing
heading	e_ψ^2	aligned with the path
accel, jerk	a^2, $jerk^2$	comfort
rate	δ_rate^2	smooth steering
lat	$(v^2 \tan \delta / L)^2$	lateral acceleration
slack	$\Sigma(s + s^2)$ over the hard groups	every constraint is soft, its slack priced at 1e5
headway	$s_h + s_h^2$	room to stop behind a lead at the horizon's end, softer (1e2)

Constraint groups (one slack each per step):

Group	Constraint
corridor	the body's front and rear corners within [lo, hi](s) less edge_margin
movers	each of the ego's 3 circles outside each mover's circle: r_ego + r + margin (+ growth · t)
ahead	the front gap_min behind a lead's rear; behind a stop gate, with the distance to stop at a_stop at the end
behind	the rear past a passing gate (going ahead of a crossing actor) by its time
grip	$a^2 + a_lat^2 \le (\mu g)^2$; $v \le v_max$
headway	at the last step: front + gap + $(v^2 - v_lead^2) / (2\ a_stop) \le$ the lead's rear

Bounds: $v \ge -1$ m/s (not 0: an active $v \ge 0$ over many stages at standstill stalls FATROP), a in [a_min, a_max], $|jerk| \le$ jerk_max, $|\delta| \le$ delta_max, $|\delta_rate| \le$ delta_rate_max (Table 6).

5. The behaviour layer

The NMPC is local: given a gate it will stop before it or go ahead of it, but it cannot choose which, and through a circle it has no gradient to go round. The behaviour layer makes those discrete choices and lets the NMPC solve each one.

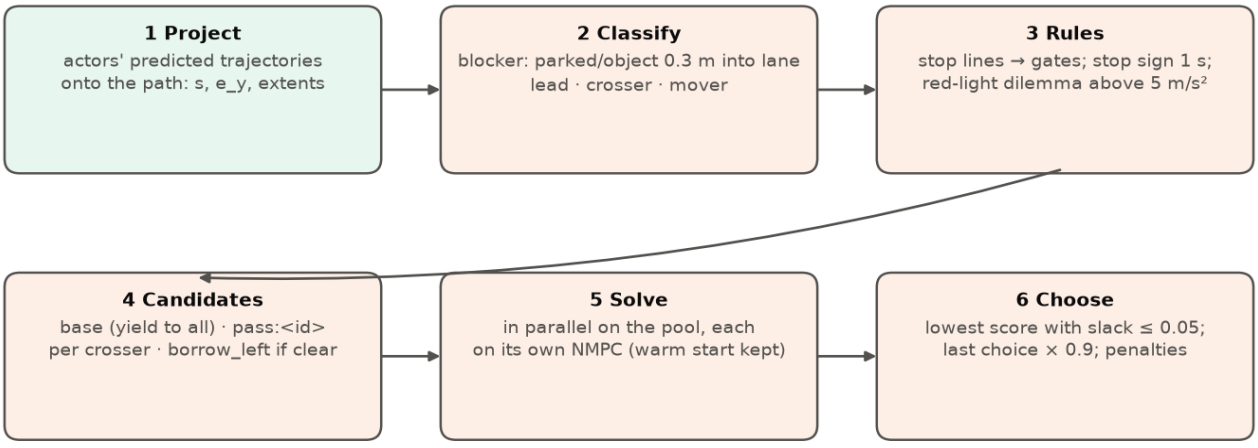


Figure 5. One behaviour step.

- **Blocker**: a parked car or an object in the lane (reaching blocker_overlap into it), kept until passed: stopped behind, or gone round in the left lane (borrow_left) when it is within blocker_range and no visible oncoming traffic comes within oncoming_gap over the manoeuvre. Once out in the other lane the overtake is committed.
- **Lead**: a vehicle in the lane ahead (or cutting in, from when it does): a hard gap and a time gap in v_ref.
- **Crosser**: a walker or a vehicle whose prediction crosses the lane: a yield gate (front stays before it over its window ± gate_buffer) in every candidate but one pass:<id> per crosser.
- **Mover**: everything else near the path: circles the NMPC keeps clear of.

- **Rules**: a red light is a stop gate unless stopping needs more than dilemma_decel; amber, more than 1.5 a_ref; a stop sign holds stop_hold at rest within 4 m of the line, then releases; the route's end is a gate.
- **Choice**: the candidate with the lowest score among those with slack ≤ slack_ok (any, if none is): its cost, plus w_out_of_lane times the squared distance from the lane centre, plus pass_penalty per crosser gone ahead of; the last choice's score × hysteresis.

6. The world model

Field	Read from CARLA	Stands in for	Limits
`actors`	world.get_actors(): vehicles, walkers, static props	3D detection + tracking	within 60 m (walkers 40 m), line of sight by ray casts from 2.5 m above the car; velocity and acceleration from the tracked boxes, not CARLA's; kept 1.5 s after last seen
`parked`	hazard lights, lane, 4 s at rest with nothing ahead	a classifier	Bench2Drive's static-mesh parked cars: a 4.8 × 2.0 m box (CARLA's boxes for them are wrong)
`predictions`	the map's lanes	a motion predictor	vehicles along their lane, the rest at constant velocity, 5 s at 0.25 s
`lanes`	map waypoints along the path	HD map / online mapping	ego lane band, left lane (width, oncoming, borrowable), every 2 m
`stops`	light stop waypoints and state, stop-sign volumes	light / sign detection	the light's state now, never its phase
`route end`	the leaderboard's sparse plan	given to every agent	–
`v_max`	hero.get_speed_limit()	sign reading	–
`car's pose`	hero.get_transform()	sensors on the car	only to report the rest in the car's frame

Never read: other actors' controls, routes or Traffic Manager targets; scenario triggers; actors out of sight; future light phases.

7. Running it

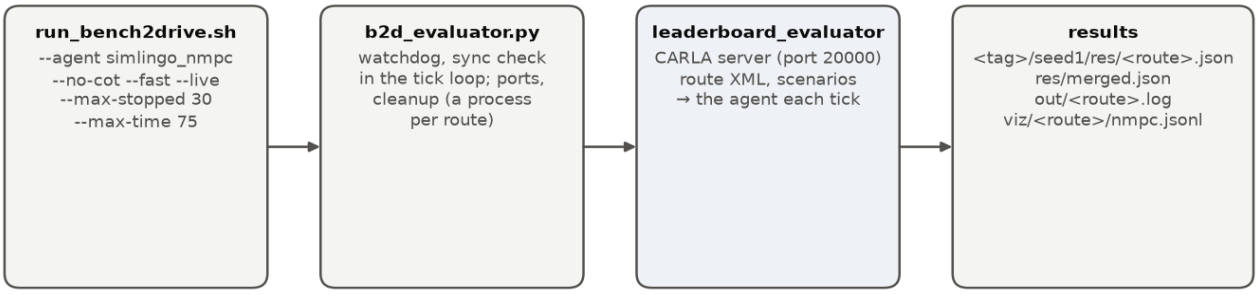


Figure 6. One route.

```
scripts/run_bench2drive.sh --agent simlingo_nmpc --routes 0-19 --no-cot --fast --live
ssh -L 8765:localhost:8765 <machine>, then http://localhost:8765
```

- One leaderboard process per route; a route with a finished result is skipped, so a stopped run carries on.
- Watchdog (b2d_evaluator.py, in the leaderboard's tick loop, outside the agent): the route is stopped and recorded as crashed once the ego has been at rest for --max-stopped seconds or after --max-time seconds of simulation time (0: off). Its score so far still counts.
- Sync check (b2d_evaluator.py): at a route's start, synchronous mode with a 0.05 s step; then every tick advancing exactly that step (a skipped or repeated frame is reported); every 10 s of simulation time, its speed against real time. Printed as "Sync: ..." lines in the route's log.
- The first NMPC in a fresh build/urban_cache compiles the solver (about a minute); later ones load it.

8. Parameters

Read from the code when this document was built.

Table 4. UrbanNMPC::Options (core/urban/include/urban/urban_nmpc.hpp)

Name	Default	Meaning
horizon	5.0	how far ahead the plan reaches [s]
h	0.2	step [s]
h_fine	0.1	step for the first `fine` seconds [s]
fine	1.0	how long the fine steps last [s]
n_movers	8	mover slots, the nearest to the plan filled
margin	0.5	clearance kept to movers [m]
edge_margin	0.15	kept between the body and the corridor's edges [m]
gap_min	0.5	Behind a lead: never closer than gap_min (a hard constraint, as for any obstacle). [m]
gap	3.0	standstill gap to a lead [m]
time_gap	1.0	time gap kept behind a lead, in the speed asked for [s]
a_stop	4.0	At the end of the horizon the car must be able to stop behind a stop gate, and behind a lead were it to brake, at this deceleration: without it a plan reaching the line just as the horizon ends never brakes. [m/s^2]
a_ref	2.0	The speed asked for tapers to standstill at stop gates and behind leads along sqrt(2 a_ref d), d the distance left: asked for the speed limit right up to them, a quadratic speed cost would rather creep towards a stop over the whole horizon than drive up to it and wait (a convex cost spreads the distance). [m/s^2]
a_lat_ref	3.0	and in bends to the speed that takes them at this lateral acceleration (grip allows more, comfort less) [m/s^2]
a_max	3.0	strongest acceleration [m/s^2]
a_min	-7.0	The strongest braking [m/s^2]; NaN: min(F_brake_max / m, mu g).
jerk_max	15.0	largest |jerk| [m/s^3]
mu	0.9	friction the plans may use
w_speed	1.0	weight of the speed cost term
w_lane	3.0	weight of the lane cost term
w_heading	1.0	weight of the heading cost term
w_accel	0.5	weight of the accel cost term
w_jerk	0.05	weight of the jerk cost term
w_rate	20.0	weight of the rate cost term
w_lat	0.3	weight of the lat cost term (lateral acceleration)
w_headway	1e2	weight of the headway cost term
w_slack	1e5	weight of the slack cost term (high enough that no violation is cheaper than braking to a stop from the speed limit)
tol	1e-3	FATROP's convergence tolerance
acceptable_tol	1e-1	Stop once the iterate has been within acceptable_tol for acceptable_iter iterations in a row.
acceptable_iter	3	iterations within acceptable_tol before stopping
max_iter	100	FATROP's iteration limit
jit	false	Compile the problem's functions to native code (with the system's C compiler): about half the time per iteration, but over a minute to build.
cache_dir	–	where the compiled solver is kept (the agent: build/urban_cache)
warm_start	true	start from the previous plan shifted; false: from the state carried on at its speed

The agent sets a_min = −7.0, jit = true, cache_dir = build/urban_cache.

Table 5. Behaviour::Options (core/urban/include/urban/behaviour.hpp)

Name	Default	Meaning
pool	4	NMPC instances, the most candidates solved at once
look_ahead	80.0	actors and stops further along the route than this are left out [m]
gate_margin	1.0	kept between the ego's body and a crosser's [m]
gate_buffer	1.0	a crosser's window widened by this much either side [s]

Name	Default	Meaning
blocker_range	40.0	a blocker nearer than this may be overtaken [m]
blocker_overlap	0.3	A parked car or an object is in the lane (a blocker, or a lead) only if it reaches this far into it; one that only grazes the lane is kept clear of as a mover, by nudging within the lane. [m]
static_margin	0.2	Kept to parked cars and objects instead of the NMPC's margin, without the growth with prediction time a moving actor's clearance has: nothing about where they will be is uncertain. [m]
oncoming_gap	20.0	kept to oncoming traffic when overtaking [m]
stop_hold	1.0	[s] at a stop sign
dilemma_decel	5.0	a light turning red that needs more than this to stop for is driven through [m/s^2]
hysteresis	0.9	the last choice's score is multiplied by this
w_out_of_lane	0.2	weight of the out_of_lane cost term (on the squared distance from the ego lane's centre, in the score)
pass_penalty	300.0	added to the score for each crosser gone ahead of
slack_ok	0.05	a candidate with any slack above this is only chosen when none is within it

Table 6. The vehicle: config/vehicles/mkz2020.json (derivations: docs/vehicle_parameters.md)

Name	Value	Source	Meaning
m	1696 kg	carla	mass: physics_control.mass
a	1.172 m	carla	centre of gravity to front axle: front axle 1.472 m ahead of the actor origin, centre of mass 0.30 m ahead of it
b	1.689 m	carla	centre of gravity to rear axle: rear axle 1.389 m behind the actor origin, plus the centre of mass's 0.30 m
F_brake_max	13000 N	derived	peak brake force at the wheels: m x 7.7 m/s^2, full brake at 10-20 m/s (scripts/identify_ego.py); the brake torque alone (4 x 800 N m / 0.355 m) would give 5.3 m/s^2
delta_max	0.95 rad	derived	road-wheel steering angle limit above 120 km/h: road-wheel angle for steer 1 at 20 m/s (it is 1.1 at 3 m/s): the smaller, so it holds at any speed
delta_rate_max	0.6 rad/s	design	road-wheel steering rate limit: CARLA steers without a rate limit (a 0.15 s lag). At 1.5 rad/s the offline overtakes (sim/urban, a kinematic plant with lags on acceleration and steering) steer back faster than the lagged plant follows, overshoot the lane and swerve; 1.0 is borderline
g	9.81 m/s^2	constant	gravity

Body (the agent's): length 4.89 m, width 1.84 + 0.2 m (mirrors), centre of gravity 0.31 m ahead of the body's centre, 0.30 m ahead of the car's origin.

Table 7. The agent, the reference and the world model (Python constants)

Name	Value	Meaning	File
DT	0.05	[s] a tick	simlingo_nmpc_agent.py
PLAN_EVERY	2	ticks between plans (10 Hz)	simlingo_nmpc_agent.py
COM_AHEAD	0.30	the MKZ's centre of mass ahead of its origin [m] (docs/vehicle_parameters.md)	simlingo_nmpc_agent.py
WP_DT	0.25	[s] between speed waypoints (config_simlingo.py: 20 fps / (wp_dilation 1 x data_save_freq 5))	simlingo_ref.py
BEHIND	15.0	[m] the path reaches behind the car, so that actors and stops just behind it project onto it	simlingo_ref.py
SPACING	2.0	[m] between the lane continuation's points, and between lane samples	route.py
CONTINUE	160.0	[m] of lane continuation past SimLingo's last point	route.py
JOIN	10.0	[m] over which the continuation joins its lane's centre	route.py
LOOK	200.0	[m] stop lines and stop signs further from the car than this are not looked at	route.py
PARKED_CAR	(4.8, 2.0)	length, width [m]	actors.py
MAX_STOPPED	env "B2D_MAX_STOPPED"	[s] at rest before the route is stopped; 0: off	b2d_evaluator.py

Name	Value	Meaning	File
MAX_TIME	env "B2D_MAX_TIME"	[s] of simulation time before the route is stopped; 0: off	b2d_evaluator.py
STOPPED_SPEED	0.1	[m/s] under which the ego counts as at rest	b2d_evaluator.py
FIXED_DT	0.05	[s] the leaderboard's step	b2d_evaluator.py
SYNC_REPORT	10.0	[s] of simulation time between real-time reports	b2d_evaluator.py
look_range	60.0	detection range [m]	actors.py
walker_range	40.0	detection range for walkers [m]	actors.py
keep	1.5	a track is kept this long after last seen [s]	actors.py
dt	0.05	control period [s]	mkz_actuation.py
kp	0.15	PI on the acceleration error: gain	mkz_actuation.py
ki	0.3	PI: integral gain	mkz_actuation.py
i_max	1.5	PI: integral limit	mkz_actuation.py

Launcher defaults (run_bench2drive.sh): --seed 1, --tries 2, --max-stopped 30, --max-time 75, live view port 8765 (NMPC_LIVE_PORT). Offline scenario plant (sim/urban/scenarios.cpp): kinematic bicycle at 5 ms, acceleration lag 0.25 s, steering lag 0.05 s rate-limited to delta_rate_max; solves at 10 Hz, plays the plan at 20 Hz.

9. Where it stands

- Offline: 22 scenarios (11 on the scenario's road, 11 on SimLingo-like local paths rebuilt each plan) drive correctly; parked_car_overtake (both) misses the 50 ms p95 solve-time check at about 52–59 ms.
- Bench2Drive route 0 (Town12, ParkingCutIn), latest run: 27.1 % route completion, no collisions, red lights, stop-sign or lane infractions; stopped by the watchdog after 30 s at rest behind a false blocker (the lane band came out 5.4 m wide, likely from the ±6-sample union in route.py's lanes). Solve time p50 32 ms, p95 105 ms, 36 of 423 plans failed.
- Not yet: SimLingo's own overtakes into the oncoming lane are held to the lane unless borrow_left is chosen; pedestrians at constant velocity only; no junction-branch hypotheses; no emergency vehicles.